

Summary of Earlier Concepts

- A computer platform is a collection of hardware resources.
- Computer applications are developed to perform some task.
- It is inefficient for applications to be written directly for a given hardware platform
- The OS was created to give applications an easy, secure, and consistent way to use different features. It acts as a standard, simple view of resources that applications can request and use.

OS Management of Application Execution

- Resources are shared by multiple applications.
- The processor switches between applications, so each seems to be progressing.
- Processor and I/O devices are used efficiently.

Process Elements

- Two essential elements of a process are:

Program code

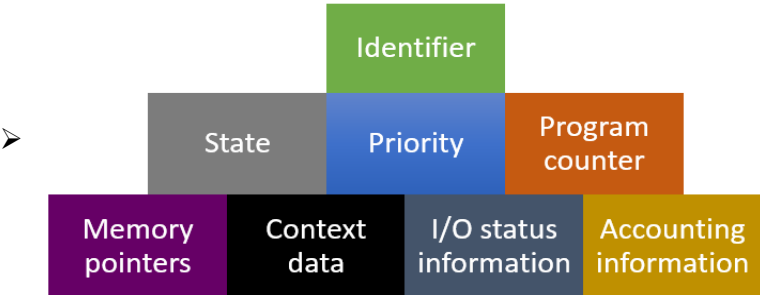
- which may be shared with other processes that are executing the same program

A set of data associated with that code

when the processor begins to execute the program code, we refer to this executing entity as a *process*



- While the program is executing, this process can be uniquely characterized by a number of elements, including:



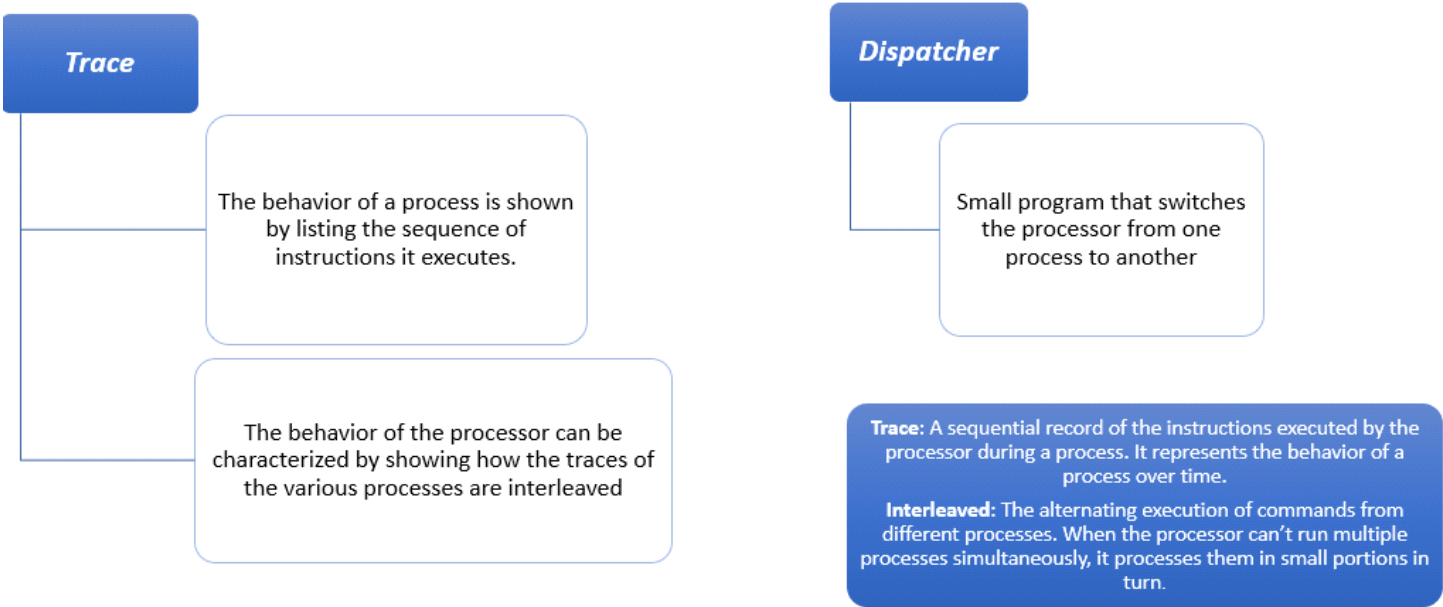
Process Control Block

Identifier
State
Priority
Program counter
Memory pointers
Context data
I/O status information
Accounting information
...

Figure 3.1 Simplified Process Control Block

- **Contains** the process elements.
- It is possible to **interrupt a running process** and later resume execution as if the interruption had not **occurred**.
- **Created and managed** by the operating system.
- Key tool that allows support for multiple processes.

! Process States !



- **Trace:** Bir işlem sırasında işlemcinin yürüttüğü komutların sıralı kaydı veya izidir. Bir sürecin zaman içindeki davranışını temsil eder.
- **Interleaved:** Farklı süreçlerin komutlarının sırayla (genelde dönüşümlü olarak) yürütülmesi anlamına gelir. İşlemcinin birden fazla süreci aynı anda yürütmesi mümkün olmadığında, süreçlerin komutları sırayla küçük parçalara bölünerek işlenir.

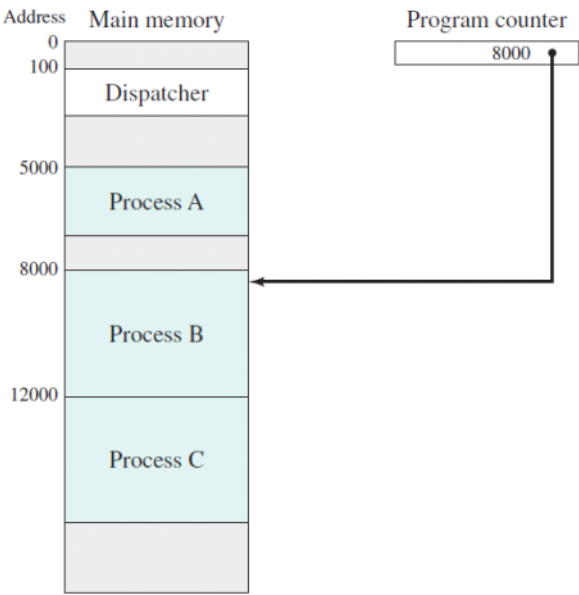


Figure 3.2 Snapshot of Example Execution (Figure 3.4) at Instruction Cycle 13

5000	8000	12000
5001	8001	12001
5002	8002	12002
5003	8003	12003
5004		12004
5005		12005
5006		12006
5007		12007
5008		12008
5009		12009
5010		12010
5011		12011

(a) Trace of process A (b) Trace of process B (c) Trace of process C

5000 = Starting address of program of process A
8000 = Starting address of program of process B
12000 = Starting address of program of process C

Figure 3.3 Traces of Processes of Figure 3.2

1	5000	27	12004
2	5001	28	12005
3	5002		
4	5003	29	100
5	5004	30	101
6	5005	31	102
		32	103
		33	104
		34	105
7	100	35	5006
8	101	36	5007
9	102	37	5008
10	103	38	5009
11	104	39	5010
12	105	40	5011
13	8000		
14	8001		
15	8002		
16	8003		
		41	100
		42	101
		43	102
		44	103
		45	104
		46	105
17	100	47	12006
18	101	48	12007
19	102	49	12008
20	103	50	12009
21	104	51	12010
22	105	52	12011
23	12000		
24	12001		
25	12002		
26	12003		

100 = Starting address of dispatcher program
Shaded areas indicate execution of dispatcher process;
first and third columns count instruction cycles;
second and fourth columns show address of instruction being executed.

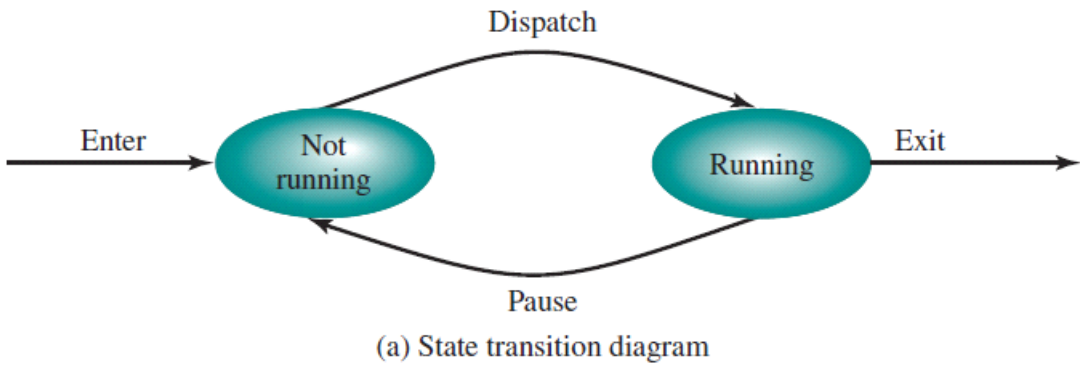
Figure 3.4 Combined Trace of Processes of Figure 3.2

Figure 3.2 shows a memory layout of three processes. To simplify the discussion, we assume no use of virtual memory; thus all three processes are represented by programs that are fully loaded in main memory. In addition, there is a small dispatcher program that switches the processor from one process to another.

Figure 3.3 shows the traces of each of the processes during the early part of their execution. The first 12 instructions executed in processes A and C are shown. Process B executes four instructions, and we assume that the fourth instruction invokes an I/O operation for which the process must wait.

Each process can run for up to six instructions before being interrupted to prevent it from using too much processor time. When process A finishes its six instructions, the dispatcher runs for a short time and then switches to process B. Process B runs but soon requests an I/O operation, so it must wait, and the processor moves to process C. This cycle continues, with the dispatcher managing each switch based on time limits or waiting states.

Two-State Process Model



The main role of the operating system is to manage process execution, including how processes share resources and determining their execution order. The simplest model classifies processes into two states: **Running and Not Running**. When a new process is created, the OS adds a process control block and sets it to the **Not Running** state, indicating it is waiting for execution. Occasionally, the **OS interrupts the currently running process, switching it to Not Running**, while **another process** is selected to run.

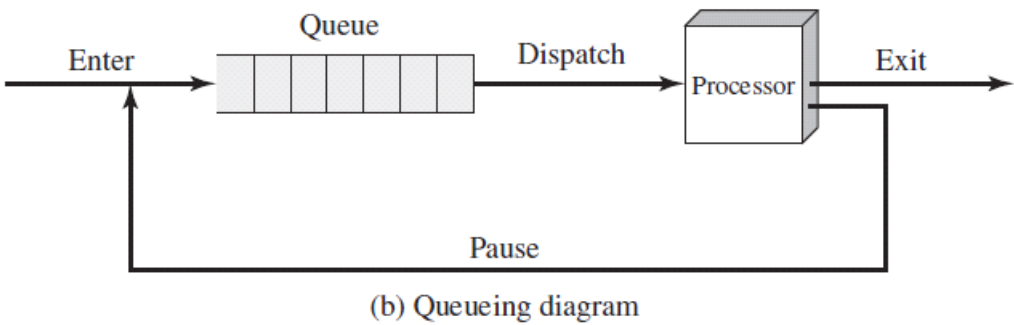
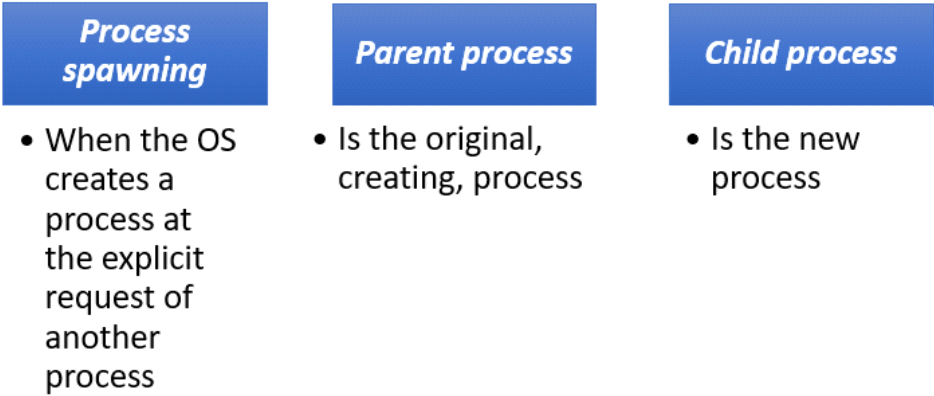


Figure 3.5 Two-State Process Model

Reasons for Process Creation

New batch job	The OS is provided with a batch job control stream, usually on tape or disk. When the OS is prepared to take on new work, it will read the next sequence of job control commands.
Interactive logon	A user at a terminal logs on to the system.
Created by OS to provide a service	The OS can create a process to perform a function on behalf of a user program, without the user having to wait (e.g., a process to control printing).
Spawned by existing process	For purposes of modularity or to exploit parallelism, a user program can dictate the creation of a number of processes.

Process Creation



Process Termination

- A process needs a way to show that it has finished. A batch job should have a HALT instruction or a clear OS service call to stop it.
- For an interactive application, the user's actions will signal when the process is done, like logging off or quitting the application.

Table 3.2 Reasons for Process Termination

Normal completion	The process executes an OS service call to indicate that it has completed running.
Time limit exceeded	The process has run longer than the specified total time limit. There are a number of possibilities for the type of time that is measured. These include total elapsed time ("wall clock time"), amount of time spent executing, and, in the case of an interactive process, the amount of time since the user last provided any input.
Memory unavailable	The process requires more memory than the system can provide.
Bounds violation	The process tries to access a memory location that it is not allowed to access.
Protection error	The process attempts to use a resource such as a file that it is not allowed to use, or it tries to use it in an improper fashion, such as writing to a read-only file.
Arithmetic error	The process tries a prohibited computation (such as division by zero) or tries to store numbers larger than the hardware can accommodate.
Time overrun	The process has waited longer than a specified maximum for a certain event to occur.
I/O failure	An error occurs during input or output, such as inability to find a file, failure to read or write after a specified maximum number of tries (when, for example, a defective area is encountered on a tape), or invalid operation (such as reading from the line printer).
Invalid instruction	The process attempts to execute a nonexistent instruction (often a result of branching into a data area and attempting to execute the data).
Privileged instruction	The process attempts to use an instruction reserved for the operating system.
Data misuse	A piece of data is of the wrong type or is not initialized.
Operator or OS intervention	For some reason, the operator or the operating system has terminated the process (e.g., if a deadlock exists).
Parent termination	When a parent terminates, the operating system may automatically terminate all of the offspring of that parent.
Parent request	A parent process typically has the authority to terminate any of its offspring.

Wop, wop, wop, wop, wop, Dot, fuck 'em up

Five-State Process Model

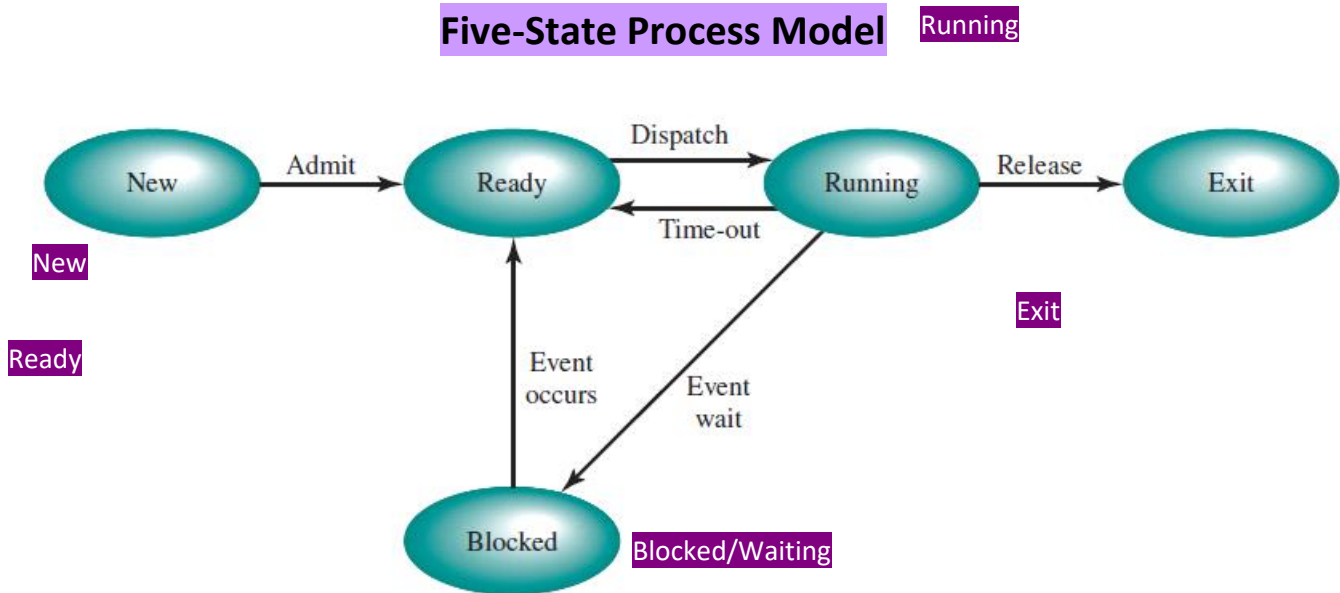


Figure 3.6 Five-State Process Model

- **Null : New**: A new process is created to execute a program.
- **New : Ready**: The OS moves a process from the New state to the Ready state when it is prepared to run.
- **Ready : Running**: The OS selects a process from the Ready state to run, which is the responsibility of the scheduler or dispatcher.
- **Running : Exit**: The currently running process is terminated by the OS if it completes its task or aborts unexpectedly.
- **Running : Ready**: A running process may return to the Ready state when it exceeds its allowed time for uninterrupted execution, a common practice in multiprogramming systems.
- **Running : Blocked**: A process enters the Blocked state when it requests a resource or action that is not immediately available, such as waiting for an I/O operation to finish.
- **Blocked : Ready**: A process in the Blocked state transitions to the Ready state when the event it was waiting for occurs, allowing it to run again.
- **Ready : Exit**: This transition is not shown in state diagrams, but a parent process can terminate a child process at any time, leading to its exit.
- **Blocked : Exit**: Similar to the previous item, a blocked process can exit if its parent terminates or if specific conditions cause it to stop.

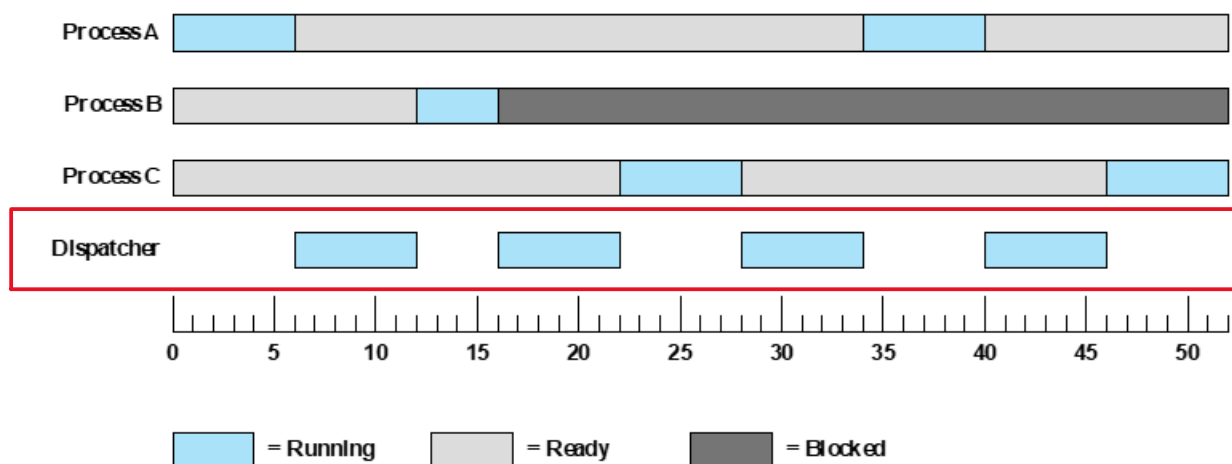


Figure 3.7 Process States for Trace of Figure 3.4

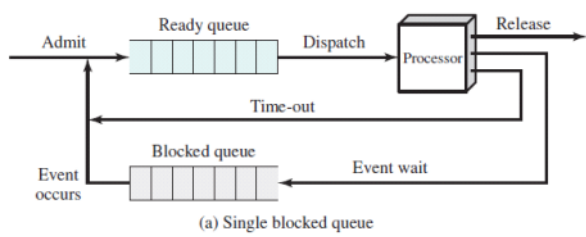
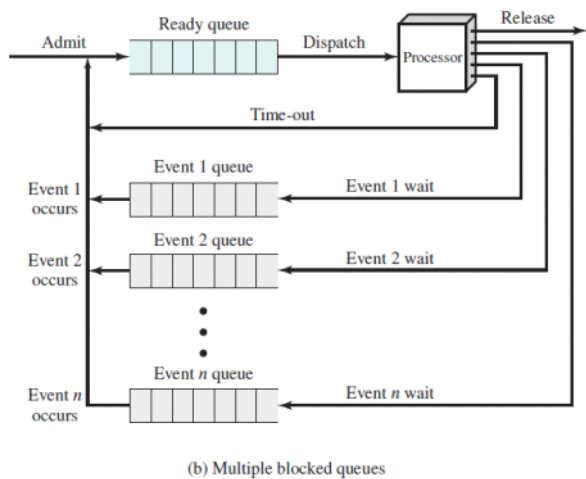


Figure 3.8a illustrates a queuing system with a **Ready queue** and a **Blocked queue**. When a process is added, it goes into the **Ready queue**. The OS selects a process from this queue to run. If a running process is removed, it is either terminated or placed back in the **Ready** or **Blocked queue**.



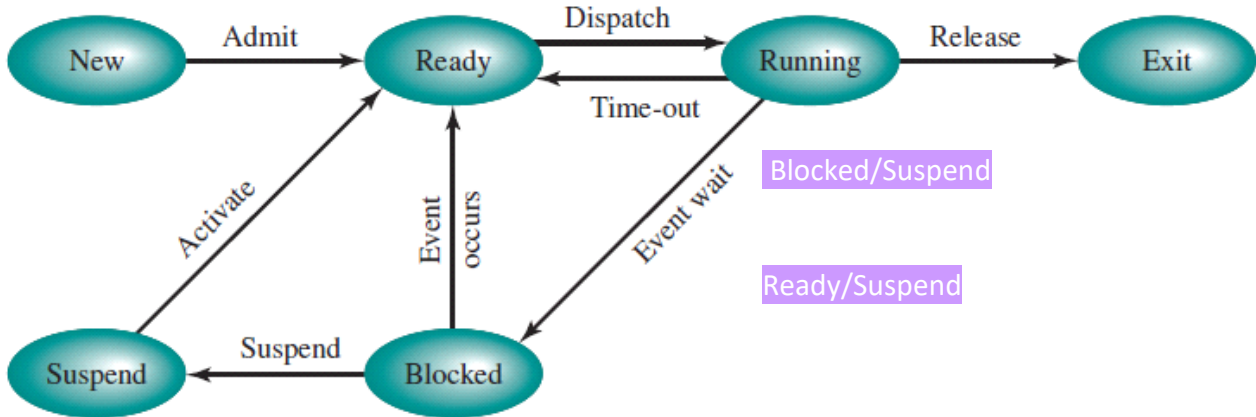
When an event occurs, processes in the **Blocked queue** waiting for that event are moved to the **Ready queue**. To improve efficiency, multiple queues can be used, as shown in **Figure 3.8b**, allowing quick transitions to the **Ready state**. Additionally, having several **Ready queues** for different priority levels enables the OS to easily identify the highest-priority process waiting the longest.

Figure 3.8 Queuing Model for Figure 3.6

! Suspended Process !

- Swapping
 - Involves moving part of all of a process from main memory to disk..
 - When none of the processes in main memory is in the Ready state, the OS swaps one of the blocked processes out on to disk into a suspend queue.
 - This is a queue of existing processes that have been temporarily kicked out of main memory, or suspended.
 - The OS then brings in another process from the suspend queue or it honors a new-process request.
 - Execution then continues with the newly arrived process.
 - Swapping, however, is an I/O operation and therefore there is the potential for making the problem worse, not better. Because disk I/O is generally the fastest I/O on a system, swapping will usually enhance performance.

Ready



Blocked/Suspend

Ready/Suspend

Blocked

(a) With one Suspend state

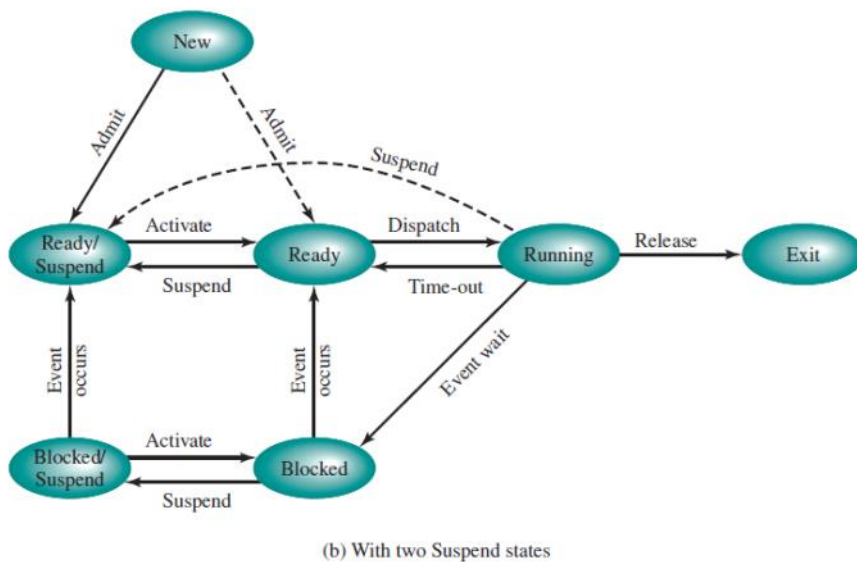


Figure 3.9 Process State Transition Diagram with Suspend States

Characteristics of a Suspended Process

- The process is not immediately available for execution.
- The process was placed in a suspended state by an agent: either itself, a parent process, or the OS, for the purpose of preventing its execution.
- The process may or may not be waiting on an event.
- The process may not be removed from this state until the agent explicitly orders the removal.

! Reasons for Process Suspension !

Swapping	The OS needs to release sufficient main memory to bring in a process that is ready to execute.
Other OS reason	The OS may suspend a background or utility process or a process that is suspected of causing a problem.
Interactive user request	A user may wish to suspend execution of a program for purposes of debugging or in connection with the use of a resource.
Timing	A process may be executed periodically (e.g., an accounting or system monitoring process) and may be suspended while waiting for the next time interval.
Parent process request	A parent process may wish to suspend execution of a descendant to examine or modify the suspended process, or to coordinate the activity of various descendants.

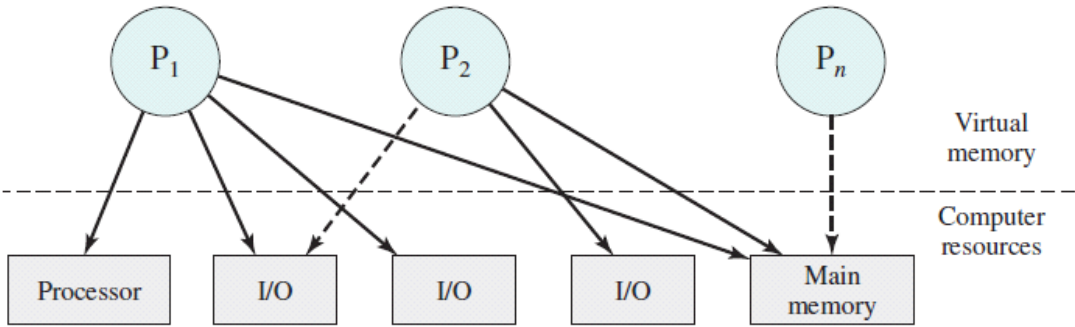


Figure 3.10 Processes and Resources (resource allocation at one snapshot in time)

In the figure, process P_1 is running; at least part of the process is in main memory, and it has control of two I/O devices. Process P_2 is also in main memory but is blocked waiting for an I/O device allocated to P_1 . Process P_n has been swapped out and is therefore suspended.

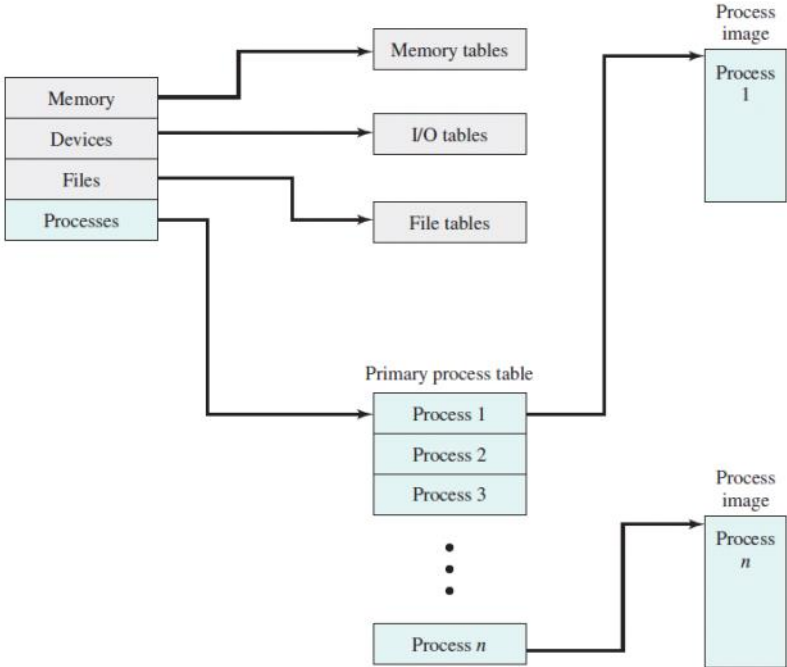


Figure 3.11 General Structure of Operating System Control Tables

Memory Tables !!

- Used to **keep track** of both main (real) and secondary (virtual) memory.
- Processes are maintained on secondary memory using some sort of virtual memory or simple swapping mechanism.

Must include:

The allocation of main memory to processes

The allocation of secondary memory to processes

Protection attributes of blocks of main or virtual memory

Any information needed to manage virtual memory



I/O Tables

- I/O tables are used by the OS to manage the I/O devices and channels of the computer system.
- At any given time, an I/O device may be available or assigned to a particular process

If an I/O operation is in progress, the OS needs to know:

- The status of the I/O operation
- The location in main memory being used as the source or destination of the I/O transfer



File Tables

These tables provide information about:

- Existence of files
- Location on secondary memory
- Current status
- Other attributes



Process Tables

- Must be maintained to manage processes.
- There must be some reference to memory, I/O, and files, directly or indirectly.
- The tables themselves must be accessible by the OS and therefore are subject to memory management.

Process Control Structures

To manage and control a process, the OS must know the necessary information about the process:

- Where the process is located
- The attributes of the process that are necessary for its management

Process Location

- A **process must include a program** or set of programs **to be executed**.
- **Memory requirements:** A process requires **enough memory to store its programs and data**.
- **Using a stack:** When a program runs, it usually uses a **stack** to manage procedure calls and pass parameters between procedures.

Process Attributes

- **Process attributes:** Each process has several attributes that the operating system uses to control it.
- **Process image:** The combination of the program, data, stack, and attributes is called the **process image**.
- **Memory management:** The location of the process image depends on the memory management system being used.

Process Identification

- **Unique identifier:** Each process gets a **unique number** called a process identifier.
- **Mapping:** The operating system needs a way to find the correct information based on the process identifier.
- **Cross-referencing:** The OS uses process identifiers in tables to **connect** and refer **to different process information**.
- **Memory tables:** Memory tables **show a map of main memory** and show which process is using each part.
- **I/O and file tables:** Similar maps are also used for input/output and file tables.
- **Communication:** When processes talk to each other, the process identifier tells the OS where to send the message.
- **Process creation:** When processes can create other processes, the identifiers show which process is the parent and which are the child processes.

Processor State Information

PSI
consists
of the
contents
of
processor
registers

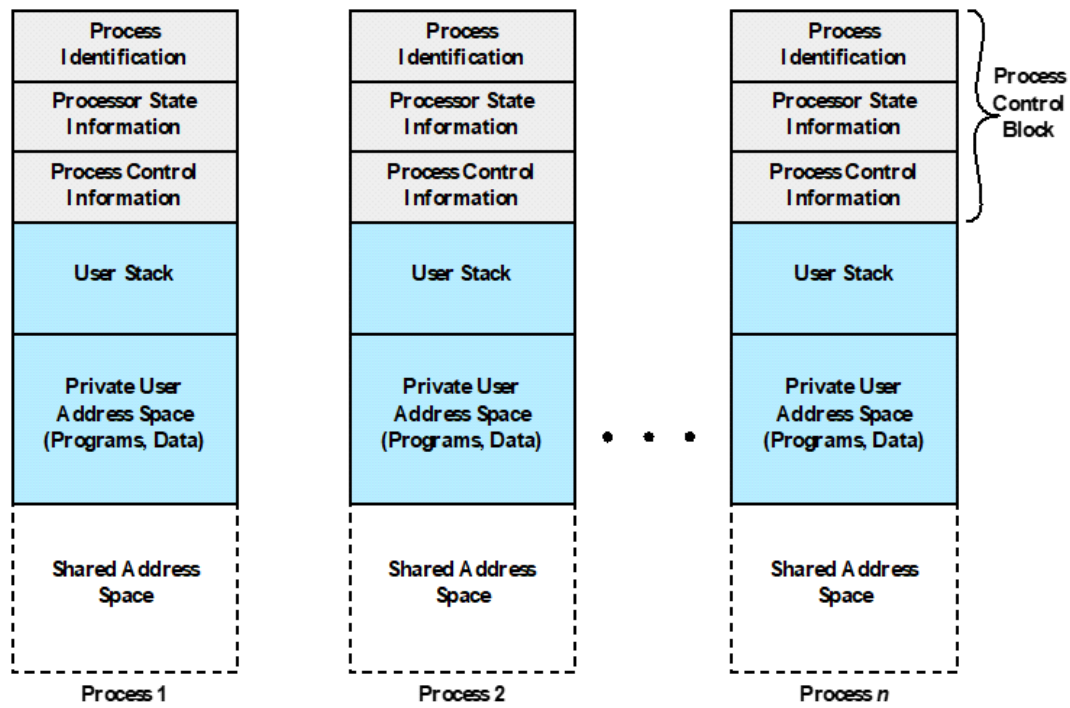
- User-visible registers
- Control and status registers
- Stack pointers

Program
status
word
(PSW)

- Contains condition codes plus other status information
- EFLAGS register is an example of a PSW used by any OS running on an x86 processor

Process Control Information

- The additional information needed by the OS to control and coordinate the various active processes



System Interrupts

Interrupt

- Due to some sort of event that is external to and independent of the currently running process
 - Clock interrupt
 - I/O interrupt
 - Memory fault
- Time slice
 - The maximum amount of time that a process can execute before being interrupted

Trap

- An error or exception condition generated within the currently running process
- OS determines if the condition is fatal
 - Moved to the Exit state and a process switch occurs
 - Action will depend on the nature of the error the design of the OS